SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Pratik Akrur Pandit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	16/10/2026

2. Practical Details

Practical No.	6
Practical Title	Different Types of Inheritance
Date Performed	16/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

To study and implement different types of inheritance in C++, including single, multilevel, multiple, hierarchical, and hybrid inheritance. The program demonstrates how derived classes inherit data and functions from base classes, promoting code reusability and supporting object-oriented programming.

4. Objectives

1. To understand the concept of inheritance and the relationship between base and derived classes.
2. To implement single, multilevel, multiple, and hierarchical inheritance in C++.
3. To demonstrate hybrid inheritance and understand the role of virtual base classes in avoiding duplicate baseclass instances.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain the concept and advantages of inheritance in object-oriented programming.
2. Develop C++ programs using different types of inheritance.
3. Understand ambiguity in multiple inheritance and use virtual base classes to resolve the diamond problem.

6. Theory / Concept

Inheritance: Inheritance is an object-oriented programming feature in which a derived class acquires the properties and functions of a base class. It improves code reusability and reduces code duplication.

1. Single Inheritance: A derived class inherits from one base class.
2. Multilevel Inheritance: A class inherits from a derived class, forming a chain of inheritance.
3. Multiple Inheritance: A derived class inherits from two or more base classes.
4. Hierarchical Inheritance: Two or more derived classes inherit from the same base class.
5. Hybrid Inheritance: Hybrid inheritance combines two or more types of inheritance. Virtual base classes can be used to prevent duplicate copies of a common base class in diamond-shaped inheritance.

In C++, inheritance is specified using a colon followed by an access specifier and the base-class name. Public inheritance generally preserves the public interface of the base class in the derived class.

7. Algorithm / Pseudocode

START

Step 1: Include the required header file and define the base classes with their member functions.

Step 2: Create derived classes to demonstrate single and multilevel inheritance.

Step 3: Define classes that inherit from multiple base classes to demonstrate multiple inheritance.

Step 4: Create multiple derived classes from a common base class to demonstrate hierarchical inheritance.

Step 5: Implement hybrid inheritance using virtual base classes, create objects, and display the inherited functions and their outputs.

STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Visual Studio Code
Libraries / Packages	iostream

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**

Branch Used	**main**
Commit ID / Latest Commit	**6673f5b1648f0de73ea171df716c31c2c08513c0**
Date of Last Commit	**16/10/2026**

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC-1	Create Dog object	Animal eats; Dog barks	Animal eats; Dog barks	Pass
TC-2	Create Puppy object	Animal eats; Dog barks; Puppy whines	Animal eats; Dog barks; Puppy whines	Pass
TC-3	Create Machine object	Printing; Scanning	Printing; Scanning	Pass
TC - 4	Create Circle and Rectangle objects	Circle and Rectangle display their shapes	Circle and Rectangle display their shapes	Pass
TC - 5	Create TeachingAssistant object	Person details displayed once	Person details displayed once	Pass

11. Results / Observations

Mention the important results obtained during execution.

The program was successfully implemented using a class and object in C++. The member functions were used to accept and display student information. The program correctly displayed the result based on the marks entered.

Screenshots / Graphs / Output:

Practical_assignment_6.cpp > Animal

```cpp

#include <iostream>
using namespace std;

// Single Inheritance
class Vehicle {
public:
    void start() {
        cout << "Vehicle starts" << endl;
    }
};

class Car : public Vehicle {
public:
    void drive() {
        cout << "Car is moving" << endl;
    }
};

// Multilevel Inheritance
class SportsCar : public Car {
public:
    void turbo() {
        cout << "Turbo mode activated" << endl;
    }
};

// Multiple Inheritance
class Camera {
public:
    void takePhoto() {
        cout << "Photo captured" << endl;
    }
};

class MusicPlayer {
public:
    void playMusic() {
        cout << "Music is playing" << endl;
    }
};

class SmartDevice : public Camera, public MusicPlayer {
};

// Hierarchical Inheritance
class Animal {
public:
```

```
48  public:
49      void category() {
50          cout << "Animal category" << endl;
51      }
52  };
53
54  class Cat : public Animal {
55  public:
56      void meow() {
57          cout << "Cat says meow" << endl;
58      }
59  };
60
61  class Cow : public Animal {
62  public:
63      void moo() {
64          cout << "Cow says moo" << endl;
65      }
66  };
67
68  // Hybrid Inheritance
69  class Account {
70  public:
71      void accountInfo() {
72          cout << "Account information shown" << endl;
73      }
74  };
75
76  class Teacher : virtual public Account {
77  };
78
79  class Researcher : virtual public Account {
80  };
81
82  class Professor : public Teacher, public Researcher {
83  };
84
85  int main() {
86      cout << "Single Inheritance" << endl;
87      Car c;
88      c.start();
89      c.drive();
90
91      cout << "\nMultilevel Inheritance" << endl;
92      SportsCar s;
```

```
91      cout << "\nMultilevel Inheritance" << endl;
92      SportsCar s;
93      s.start();
94      s.drive();
95      s.turbo();
96
97      cout << "\nMultiple Inheritance" << endl;
98      SmartDevice d;
99      d.takePhoto();
100     d.playMusic();
101
102     cout << "\nHierarchical Inheritance" << endl;
103     Cat cat;
104     Cow cow;
105     cat.category();
106     cat.meow();
107     cow.category();
108     cow.moo();
109
110     cout << "\nHybrid Inheritance" << endl;
111     Professor prof;
112     prof.accountInfo();
113
114     return 0;
115 }
116
```

Output :

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
Single Inheritance
Vehicle starts
Car is moving

Multilevel Inheritance
Vehicle starts
Car is moving
Turbo mode activated

Multiple Inheritance
Photo captured
Music is playing

Hierarchical Inheritance
Animal category
Cat says meow
Animal category
Cow says moo

Hybrid Inheritance
Account information shown
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

In this practical, different types of inheritance in C++ were implemented and studied successfully. Single, multilevel, multiple, hierarchical, and hybrid inheritance were demonstrated using suitable classes and objects. The practical helped in understanding code reusability, relationships between classes, and the use of virtual base classes to avoid ambiguity in hybrid inheritance.

13. Assignment Questions

Q1. _____

Answer:

_____

_____

_____

Q2. _____

Answer:

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Pratik Akrur Pandit	202501110116	